





SESIÓN 09:

DIAGRAMAS DE CLASES Y SUS RELACIONES

RELACIÓN BINARIA Y REFLEXIVA

Docente del Curso



Reflexión



Puedes ver el video:

https://www.youtube.com/watch?v=ZClzboNlvGM&ab_channel=Jes%C3%BAsAlbertMagro



Al terminar la sesión el estudiante desarrolla y expone casos de la programación orientada a objetos aplicando Diagramas de clases y sus relaciones binaria y reflexiva, mediante el uso del lenguaje de programación de Java y con la asesoría brindada por el docente.



los diagramas de clases son una forma de representar las clases en un sistema de software y las relaciones entre ellas.

Hay varios tipos de relaciones entre clases, incluyendo asociación, agregación, composición, herencia, y dependencia.

Una relación binaria se refiere a una relación entre dos clases, y una relación reflexiva es aquella en la que una clase se relaciona consigo misma.

Elementos Clave de un Diagrama



1

Clases

Representan los objetos del mundo real o conceptos.

2

Atributos

Características o propiedades de una clase.

3

Métodos

Acciones u operaciones que puede realizar una clase.



Definición

Relación entre dos clases diferentes.
diferentes.

Ejemplos

Un estudiante se inscribe en un
curso.

Casos de Uso

Modelar interacciones entre entidades distintas.



¿Qué es?

Una clase se relaciona consigo misma.



¿Cuándo se utiliza?

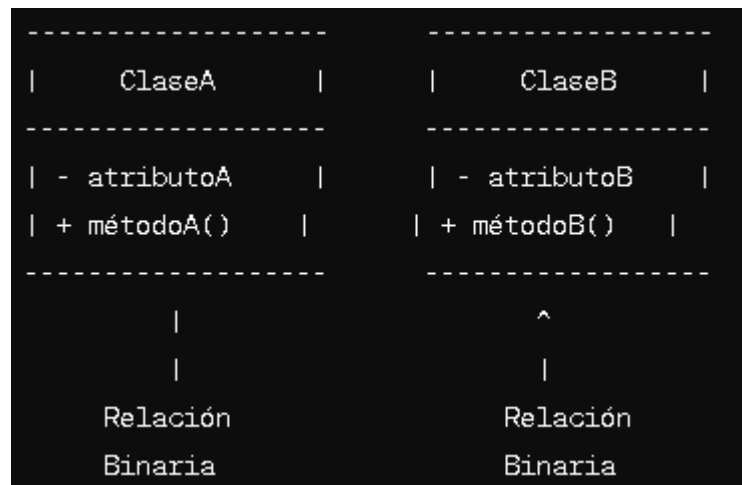
Cuando una entidad tiene relación con otras del mismo tipo.

CONSOLIDANDO RELACIÓN BINARIA Y REFLEXIVA



Una relación binaria y reflexiva en Java se puede representar a través de diagramas de clases y sus relaciones.

Una relación binaria y reflexiva significa que cada objeto de una clase está relacionado consigo mismo y posiblemente con otros objetos de la misma clase.



```

public class ClaseA {
    private int atributoA;

    public void metodoA() {
        // Código del método A
    }

    // Otros métodos y atributos
}

// Clase B
public class ClaseB {
    private int atributoB;

    public void metodoB() {
        // Código del método B
    }

    // Otros métodos y atributos
}

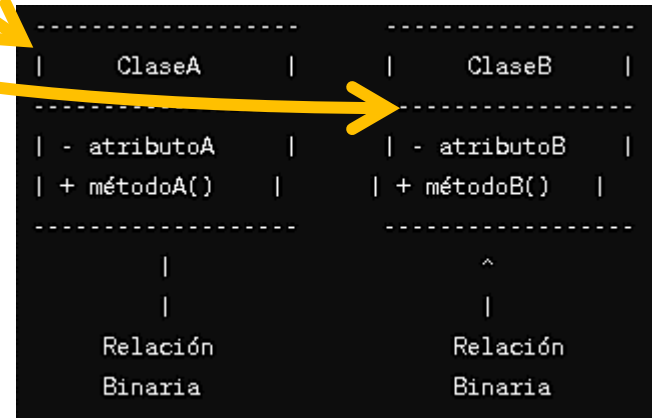
// Relación binaria entre ClaseA y ClaseB
public class RelacionBinaria {
    private ClaseA objetoA;
    private ClaseB objetoB;

    // Constructor y métodos para interactuar con las clases
}

// Relación reflexiva en una clase
public class RelacionReflexiva {
    private RelacionReflexiva mismaClase;

    // Constructor y métodos para interactuar con la misma c
}

```



EJEMPLO

```
// Clase que representa un objeto en la relación binaria y reflexiva
public class Objeto {
    private int id;
    private String nombre;

    public Objeto(int id, String nombre) {
        this.id = id;
        this.nombre = nombre;
    }

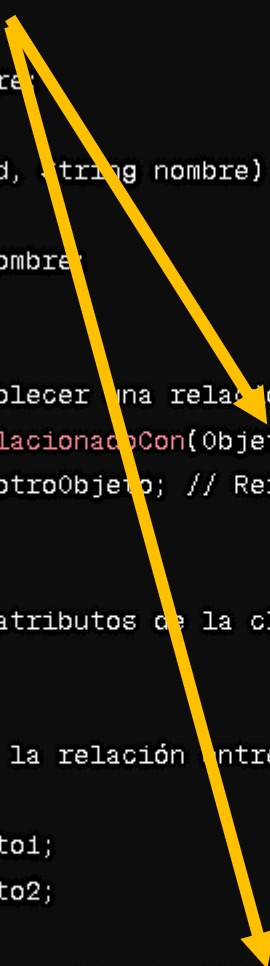
    // Método para establecer una relación reflexiva con otro objeto de la misma cl
    public boolean esRelacionadoCon(Objeto otroObjeto) {
        return this == otroObjeto; // Reflexividad
    }

    // Otros métodos y atributos de la clase
}

// Clase que representa la relación entre objetos
public class Relacion {
    private Objeto objeto1;
    private Objeto objeto2;

    public Relacion(Objeto objeto1, Objeto objeto2) {
        this.objeto1 = objeto1;
        this.objeto2 = objeto2;
    }

    // Otros métodos y atributos de la clase
}
```

Two yellow arrows originate from the 'Objeto' class definition. One arrow points to the 'esRelacionadoCon' method, and the other points to the 'Relacion' class definition, indicating a relationship or dependency between the two classes.

El ejemplo en Diagrama de Clases



```
+-----+
| Objeto |
+-----+
| - id: int |
| - nombre: String |
+-----+
| + Objeto(id: int, nombre: String) |
| + esRelacionadoCon(otroObjeto: Objeto): boolean |
+-----+
```

```
+-----+
| Relacion |
+-----+
| - objeto1: Objeto |
| - objeto2: Objeto |
+-----+
```



1

Claridad

Utilizar nombres descriptivos para clases y atributos.

2

Simplicidad

Evitar diagramas demasiado complejos.

3

Consistencia

Seguir las convenciones de UML.

CONCLUSIONES Y PREGUNTAS

